

Experiment 04: Interfacing Sensor Peripherals

Roll Number: 21BIT191

Objective: To interface DHT11 digital temperature sensors with the Raspberry Pi to get temperature and humidity readings.

Equipment and Components: Raspberry Pi – 4B, SD card/ USB Drive, Laptop, SD card reader (optional), jumper wires, DHT11 Sensor.

Theory: DHT11 and DHT22 are popular digital sensors used to measure temperature and relative humidity. These sensors are widely favored in various IoT applications due to their simplicity and ease of use with microcontrollers like the Raspberry Pi. Both sensors have an integrated chip that converts the analog signals from the environment into digital output, which can then be easily processed by a microcontroller.

DHT11:

- Temperature range: 0 to 50 °C with an accuracy of ± 2 °C.
- Humidity range: 20 to 90% with an accuracy of $\pm 5\%$.
- Sampling period: 1 second.

DHT22:

- Temperature range: -40 to 80 °C with an accuracy of ± 0.5 °C.
- Humidity range: 0 to 100% with an accuracy of $\pm 2\%$.
- Sampling period: 2 seconds.

While both sensors are capable of measuring temperature and humidity, the DHT22 offers greater accuracy and a wider range for both parameters compared to the DHT11. However, the DHT22 is also more expensive and has a slower sampling rate.

Working Principle of DHT Sensors

The DHT sensors contain a thermistor and a capacitive humidity sensor to measure temperature and humidity, respectively. The integrated chip converts the analog signals from these components into digital data, which is then transmitted to the Raspberry Pi through a single-wire communication protocol.

The single-wire protocol simplifies the wiring, making it easier to integrate these sensors into IoT projects. The communication between the sensor and the Raspberry Pi involves the sensor sending a digital signal that represents the current temperature and humidity.

Interfacing with Raspberry Pi

To interface the DHT11 or DHT22 with the Raspberry Pi, three connections are required:

1. **VCC:** Connected to the 3.3V pin of the Raspberry Pi.
2. **Data:** Connected to any GPIO pin on the Raspberry Pi (commonly GPIO 4). A pull-up resistor ($4.7k\Omega$) may be required to stabilize the signal.
3. **GND:** Connected to the ground pin of the Raspberry Pi.

By using the **Adafruit_CircuitPython_DHT** library, Python code can be written to read the temperature and humidity data from the sensor. The library simplifies the interaction with the sensor by providing easy-to-use methods to retrieve temperature and humidity values.

Python Script for Reading Data

The Python script uses the Adafruit library to initialize the sensor on a specified GPIO pin and continuously reads the temperature and humidity data. The data is then printed in both Celsius and Fahrenheit, along with the humidity percentage. The script also includes error handling to manage any communication issues with the sensor.

Application

The ability to measure temperature and humidity is critical in various applications, including environmental monitoring, HVAC systems, and smart home automation. By interfacing DHT sensors with the Raspberry Pi, real-time environmental data can be collected and analyzed, enabling more responsive and intelligent systems.

Procedure:

1. After setting up raspberry pi in experiment 2, now you can direct open PuTTY and repeat same steps as done previously. So, the Raspberry Pi OS desktop will appear.
2. Now, we need to update our Raspberry Pi. Run the following command to update and upgrade your Raspberry Pi to ensure that you have the latest software and libraries:

```
sudo apt update && sudo apt upgrade
```

If there are updates available, the system will prompt you to confirm. Press Y and Enter to proceed with the update. This process may take a few minutes.

3. We need to make an Virtual Environment for the Python. Navigate to your Desktop directory using the Terminal:
`cd ~/Desktop`

Create a new folder for your project. This folder will contain your virtual environment and the Python code:

```
mkdir dht_test
```

Move into the newly created directory:

```
cd ~/Desktop/dht_test
```

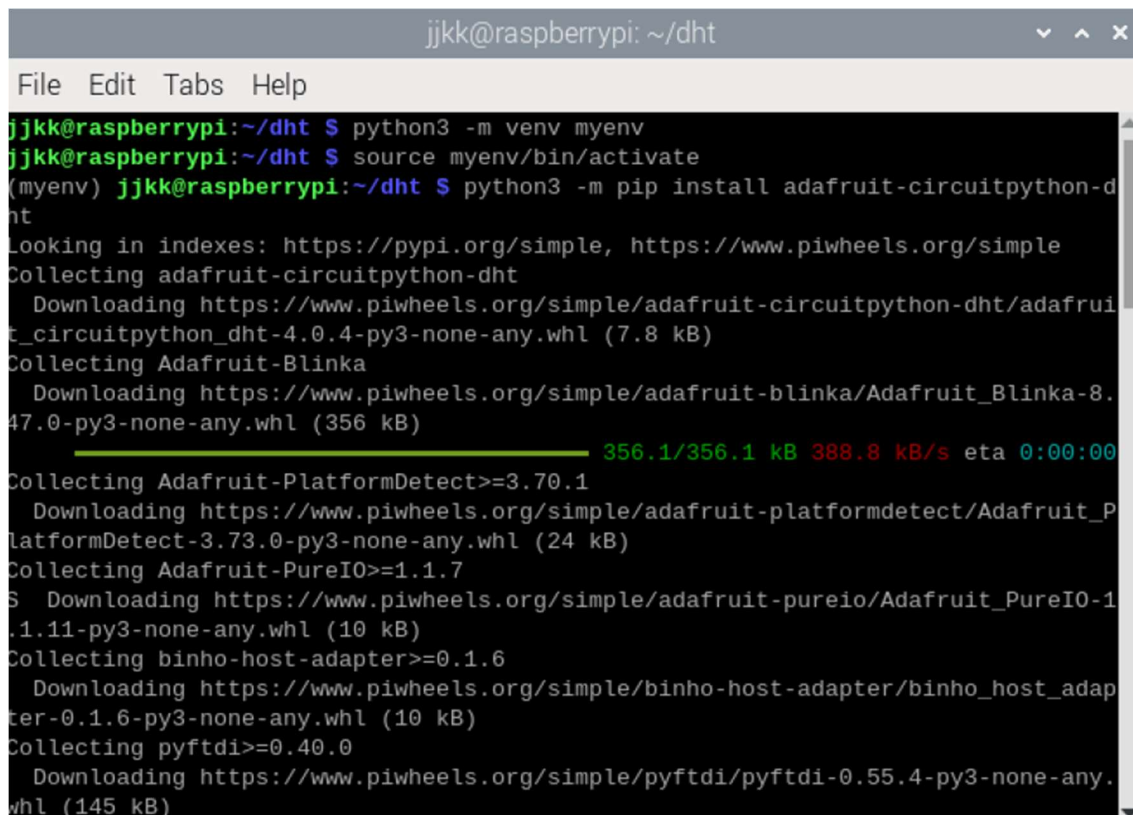
Create a virtual environment within this directory named myenv (or any other name you prefer):

```
python3 -m venv myenv
```

Activate the virtual environment:

```
source myenv/bin/activate
```

After activation, your terminal prompt will change, indicating that you are now working within the virtual environment.



```
jjkk@raspberrypi: ~/dht
File Edit Tabs Help
jjkk@raspberrypi:~/dht $ python3 -m venv myenv
jjkk@raspberrypi:~/dht $ source myenv/bin/activate
(myenv) jjkk@raspberrypi:~/dht $ python3 -m pip install adafruit-circuitpython-dht
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting adafruit-circuitpython-dht
  Downloading https://www.piwheels.org/simple/adafruit-circuitpython-dht/adafruit_circuitpython_dht-4.0.4-py3-none-any.whl (7.8 kB)
Collecting Adafruit-Blinka
  Downloading https://www.piwheels.org/simple/adafruit-blinka/Adafruit_Blinka-8.47.0-py3-none-any.whl (356 kB)
356.1/356.1 kB 388.8 kB/s eta 0:00:00
Collecting Adafruit-PlatformDetect>=3.70.1
  Downloading https://www.piwheels.org/simple/adafruit-platformdetect/Adafruit_PlatformDetect-3.73.0-py3-none-any.whl (24 kB)
Collecting Adafruit-PureIO>=1.1.7
  Downloading https://www.piwheels.org/simple/adafruit-pureio/Adafruit_PureIO-1.1.11-py3-none-any.whl (10 kB)
Collecting binho-host-adapter>=0.1.6
  Downloading https://www.piwheels.org/simple/binho-host-adapter/binho_host_adapter-0.1.6-py3-none-any.whl (10 kB)
Collecting pyftdi>=0.40.0
  Downloading https://www.piwheels.org/simple/pyftdi/pyftdi-0.55.4-py3-none-any.whl (145 kB)
```

4. Now, we need to install the library in the virtual environment. Install the Adafruit CircuitPython DHT Library.

With the virtual environment active, install the Adafruit CircuitPython DHT library by running the following command:

```
python3 -m pip install adafruit-circuitpython-dht
```

This library will allow your Raspberry Pi to interface with the DHT11/DHT22 sensor and retrieve temperature and humidity readings.

5. Then Write the Python Script. In the same directory where you created the virtual environment (`~/Desktop/dht_test`), create a new Python file, for example, `dht_basic.py`. Open the file in a text editor and copy the following Python code into it:

```

1  import time
2  import board
3  import adafruit_dht
4
5  sensor = adafruit_dht.DHT11(board.D4)
6
7  while True:
8      try:
9          # Print the values to the serial port
10         temperature_c = sensor.temperature
11         temperature_f = temperature_c * (9 / 5) + 32
12         humidity = sensor.humidity
13         print("Temp={0:0.1f} C, Temp={1:0.1f} F, Humidity={2:0.1f}%".format(temperature_c, temperature_f, humidity))
14
15     except RuntimeError as error:
16         # Errors happen fairly often, DHTs are hard to read, keep going
17         print(error.args[0])
18         time.sleep(2.0)
19         continue
20     except Exception as error:
21         sensor.exit()
22         raise error
23
24     time.sleep(3.0)
25
26

```

Save the file and close the editor.

6. Further, Run the Python Script. Ensure that the virtual environment is active. If it's not, reactivate it by running: `source myenv/bin/activate`

Run the Python script using the following command: `python dht_basic.py`

Observe the output in the Terminal. The temperature (in both Celsius and Fahrenheit) and humidity readings should be displayed every three seconds.

```

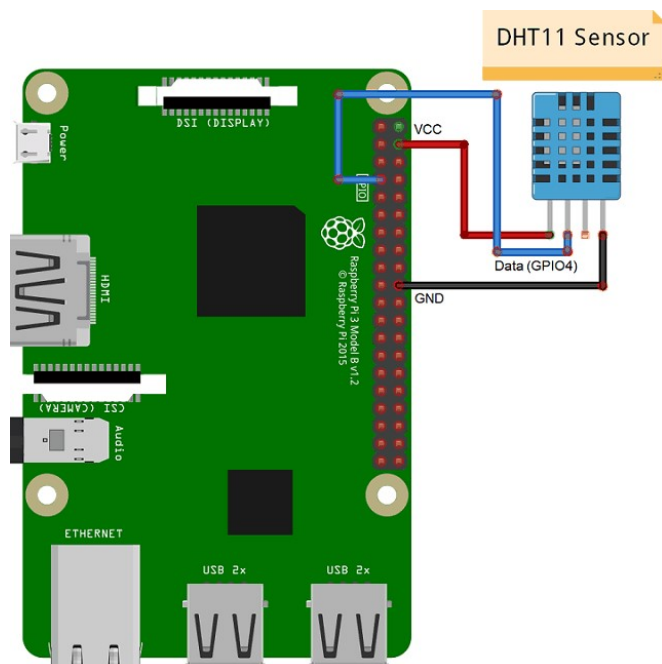
jjkk@raspberrypi: ~/dht/myenv
File Edit Tabs Help
it-circuitpython-requests-4.1.6 adafruit-circuitpython-typing-1.11.0 binho-host-
adapter-0.1.6 pyftdi-0.55.4 pyserial-3.5 pyusb-1.2.1 rpi-ws281x-5.0.0 sysv-ipc-1
.1.0 typing-extensions-4.12.2
(myenv) jjkk@raspberrypi:~/dht $ python dhts.py
python: can't open file '/home/jjkk/dht/dhts.py': [Errno 2] No such file or dire
ctory
(myenv) jjkk@raspberrypi:~/dht $ ls
myenv
(myenv) jjkk@raspberrypi:~/dht $ cd myenv
(myenv) jjkk@raspberrypi:~/dht/myenv $ ls
bin dhts.py include lib lib64 pyenv.cfg
(myenv) jjkk@raspberrypi:~/dht/myenv $ python dhts.py
Temp=29.0 C, Temp=84.2 F, Humidity=61.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Temp=28.0 C, Temp=82.4 F, Humidity=59.0%
Checksum did not validate. Try again.

```

Result:



Block Diagram:



Discussion: During this experiment, the Raspberry Pi was successfully set up to read temperature and humidity data from a DHT11 sensor using the Adafruit CircuitPython DHT library. The procedure involved creating a virtual environment, installing the required Python libraries, and running a Python script to read the sensor data.

The script initially encountered a common issue where the DHT sensor readings could not be validated, as indicated by the "Checksum did not validate. Try again." error message. This is typical when working with DHT sensors, as they can be sensitive to timing and environmental conditions. Despite this, the sensor consistently provided temperature and humidity readings, although with occasional errors that the script was designed to handle gracefully.

The successful readings show that the DHT11 sensor is functioning correctly within its expected accuracy range. The temperature and humidity data were displayed in both Celsius and Fahrenheit, which is useful for verifying the sensor's performance under different conditions.

Conclusion: The experiment demonstrated the process of interfacing a DHT11 sensor with a Raspberry Pi and retrieving real-time temperature and humidity data. The Raspberry Pi's ability to handle sensor data in a Python virtual environment was effectively shown, with the Adafruit CircuitPython library providing a straightforward method to interact with the DHT11 sensor.

The minor issues with checksum validation highlighted the importance of error handling in sensor-based projects, as environmental factors can influence the accuracy and reliability of data readings. Overall, the lab was successful in achieving its objectives, and the Raspberry Pi proved to be a suitable platform for sensor interfacing tasks.